Myeongseong Kim

5 years 11 months of experience

A frontend engineer who migrates legacy code without downtime and records the reasoning behind decisions as RFCs so they settle into team standards

maeng2418@gmail.com 010 8663 2663 maeng2418.github.io github.com/maeng2418 maeng2418.github.io/portfolio

Key achievements

FE

- **React conversion rate from 0% in 2022 to 98.9% in 2026**
  Gradual React conversion of the legacy Vue/Nuxt code
- **build time from 589.24s to 1.10s on cache hit**
  Introduced incremental builds
- **implemented new design components and applied VRT**
  Built components for NDSP, the partner-system design system

AI infra

- **developed 8 plugins and improved developer productivity**
  Built the in-house Claude plugin marketplace (stay-claude-plugins)
- **developed, rolled out and operated the automated PR review bot**
  Built pr-reviewer, a PR review bot on the Claude Agent SDK
- **redesigned the domain-knowledge system as an LLM-Wiki**
  Built a domain wiki with LLMs

Experience

NOL Universe

2022.09 - Present

Stay Supply FE team · FE Engineer

FE

Gradual Vue/Nuxt → React conversion

Problem Joined 3 legacy Vue/Nuxt apps, coming from a React background. 7 legacy repositories (543 items) coexisted with 10 new ones, and 3 frontend engineers had to convert them together

Decision Set the conversion procedure: UI markup → API → test cases → nginx route switch → removal of the Vue code

Action In 2023, converted hotel discount management and rate plans to React (5 repos) and resolved all 30 QA defects (2023.06); in 2024, carried out conversion round 9 (Jira 12, PRs 19)

Result React conversion rate: 0% (2022) → 30.6% (2023) → 77.7% (2024) → 93.5% (2025) → 98.9% (2026). The team's conversion rate in 2024 was 78/112 (69.6%), a team result.

Build and test foundation

Problem There were no tests at all. The monorepo imported code by direct source reference rather than as built output, so that code was rebuilt with every service deployment and builds were slow

Decision Turborepo pays off when packages import built output, but this monorepo references source directly, so chose incremental builds instead of adopting it. For end-to-end tests, chose Playwright: it supports headless browsers, which suits CI/CD, and offers tools such as codegen that make tests easy to create

Action Incremental TypeScript builds and removal of circular dependencies, introduced Playwright end-to-end tests, built a devtools + MSW code generator

Result Build time from 589.24s to 1.10s on cache hit; 2,833 lines of new end-to-end test code

Client log instrumentation DevTools

Problem 304 log definitions were scattered across per-app generated code, with no way to check them outside the code

Decision Moved the cost of checking onto the screen itself: built the inspector in vanilla DOM so the MUI and NDSP app families could share it, and limited changes to collection behavior to a single pilot screen

Action Instrumentation API (clientLogTag / useClientLogPage), DevTools (catalog, inspector, txtpb generation, auto-firing), automatic screenshot capture, infra-protobuf extensions — covering 304 log definitions and 174 files calling log senders

Result Used the measured results as the basis for follow-up work

Overseas Partner Central

Problem As the domain grew more complex, the existing feature-oriented architecture became hard to maintain

Decision Moved shared policies out of the screens into lower layers and split the FSD layers so that screens only compose them

Action Introduced steiger FSD linting and the useControllerGroup shared hook at kickoff; built the calendar and Rate Query virtual scrolling for the NOL Partner Central MVP. Rolled the conventions out in a 6-step Stacked PR

Result Normalized the whole codebase to the conventions, from 75~80% compliance at the initial survey

Tech-stack migration of the partner onboarding admin (lead)

Problem It was built on Nuxt 2, which faced end of support, and it carried server-management overhead and a lot of legacy code

Decision Wrote the [RFC] React migration strategy for the partner onboarding admin (comparing 11 alternatives) and the [TechSpec] on Docker/Kubernetes and path-based routing

Action Set up FSD, Module Federation, S3 + CloudFront, CI/CD, a Dockerfile and EKS manifests; removed a Sentry auth token that was exposed in the client bundle

Result Moving from SSR to CSR removed the need to manage servers, and the app now runs from a CDN. Moving from Nuxt to React reduced development effort and made current libraries available

NDSP components

Problem The design of the whole partner system needed to be unified, and dependencies on external libraries such as MUI had to go

Decision Modeled the interfaces on MUI and antd so that values and handlers stay consistent across components such as input fields

Action Added new NDSP components — 8 in 2025 and 5 in 2026; registered the Fieldset component for VRT

Result Tracked the services that install the library and confirmed that adoption keeps growing; VRT keeps CI/CD tests stable

Structural refactoring

Problem Sales-lock rules were scattered across 11 files, so changing one policy meant finding and editing every place that used it

Decision Separated per-channel rules into policy declarations instead of code branches

Action Gathered the scattered rules into a single declaration and left the code that uses them untouched

Result Policy change points reduced from 11 files to 1 place

Rate-plan screens

Problem Frontend work could not start until the API was finished, so frontend and backend work ran one after the other

Decision Had frontend and backend agree on the API spec first, so both could work in parallel before the API was finished

Action Wrote a Mock API ahead of the backend for the CM discount rate plans; replaced prop drilling with FormProvider and fixed an infinite re-render

Result The re-render problem was resolved

Partner center maintenance

Problem The unified numeric input component was reverted twice

Decision Wrote test cases first, then re-applied it

Action Introduced CHANGELOG.md in 4 repos; moved fonts from an external CDN to internal resources

Result A wide range of test cases kept issues to a minimum and covered many situations

Admin, integration and promotion screens

Problem CM integration and promotions were not supported, so a wider range of products could not be sold

Decision The Partner Central screens were similar or identical to the internal Partner Central admin screens, so built the admin screens first and reused them

Action 12 admin screens for external users, a 3-step extension of the CM integration lock policy, and 9 promotion screens with 6 APIs

AI infra

AI dev infrastructure — in-house Claude plugin marketplace

Problem Development tools were tied to the monorepo, the line between personal and team assets was unclear, and mandatory rules risked being missed. Team members' awareness and use of AI were also still low, so AI use needed to grow

Decision Compared 9 alternatives and adopted a 2-axis structure that separates repository-bound assets from shared ones; documented the Claude Plugin structure and distribution strategy (2026.04)

Action Split out the plugin repository stay-claude-plugins, per-repo plugins, Atlassian MCP OAuth and a team statusline; commands create-pr v2, create-jira, /deploy, design-sync, feature-flag and publish-artifact; fixed marketplace manifest and cache paths, added CI/lint/format and aligned GitHub rulesets

Result 8 plugins distributed and applied to 5 consuming repositories

stay-context knowledge plugin and LLM-Wiki

Problem With many domains, a way was needed to accumulate data into a wiki so that domain knowledge would not be lost

Decision Before moving to an LLM-Wiki, recorded 3 rejected alternatives together with the reasons for rejecting them

Action Built the domain-knowledge plugin stay-context; moved it to an LLM-Wiki with a knowledge build pipeline for the large monorepo. Designed an ADD/UPDATE/NOOP decision structure and a 2-layer guard against self-proliferation

Result 270 PRs generated automatically by the SessionEnd hook, 54 of them merged; audit score from 0.52 to 0.87, evaluator 93/85/90/92, 129 tests passing

pr-reviewer, a PR review bot

Problem Slow reviews and PR approvals were holding development back, and AI was finding defects better than people

Decision Chose the Claude Agent SDK because Claude is the team's main AI. Decided to separate the engine and the console because they run in different environments (Node with the SDK versus a Tauri webview), the GitHub token and SDK credentials had to stay inside the engine as a security boundary, the engine needed to run on its own (dry-run and tests), and a failure in one should not spread to the other

Action Built pr-reviewer, a PR review bot on the Claude Agent SDK, with the console and the engine as separate apps. The engine reviews PRs with the Claude SDK, checks that each flagged line is correct, posts drafts through gh and logs each run to SQLite; the console is a Tauri desktop app that shows a tray, approval bubbles and a dashboard, only reads the database, and lets its screens call Rust commands only through an allowlist

Result Developed, rolled out and operated pr-reviewer, the automated PR review bot on the Claude Agent SDK

AI convention assets

Problem Skills and rules, once written, could quietly turn into legacy; they needed to be kept up to date and managed

Decision Pushed the rules down into the execution layer so that they apply automatically while the work is being done

Action Glob-scoped .claude/rules, an audit of 18 CLAUDE.md files in the monorepo, ADR-to-skill conversion, /project-sync with per-domain audit agents; adopted Cursor and Claude Code for daily work in 2025 and shared an AI tooling guide with the team

Result When a rule changes, only one skill needs editing

CI/CD automation

Problem Several GitHub Actions had to be run by hand, and Slack notification messages had to be written manually

Decision Feature flags had been kept in an Excel sheet, which led to frequent typos and other fat-finger mistakes and poor change history, so decided to move them onto S3

Action 8 GitHub Actions composite actions, automatic Slack notification before swap, an S3-based feature-flag system with 13 flags removed

Result 33 Launch Plan runs

Lint and CI gates

Problem As the monorepo grew, running the work locally became heavier and slower

Decision Promoted recurring review comments to lint rules and build and type checks to CI gates

Action React hook-deps ESLint rule, blocking direct pushes to dev/master, Sentry log titles, Auto-Release automation, PR templates and production push protection (5 repos), a review-checkbox merge-control Action, and commitlint and cspell

Result New repositories are set up in their first PR, so the conventions are enforced from the first commit. Some of the work was split out and moved to GitHub Actions, which cut the time it takes

AI policy classifier

Problem Policy documents were scattered across many places, and the aim was to manage and maintain them automatically in a single spreadsheet

Decision Chose a Chrome extension so that anyone could easily extract policies straight from the scattered policy document pages

Action Developed an AI policy classifier as a Chrome extension

Result Taken on as an official Q4 project

PRs authored · PRs reviewed per year: 2022 (from joining) 45 · 22, 2023 369 (deduplicated) · 474, 2024 404 · 798, 2025 480 · 554

In 2026 (as of 2026.10): 913 PRs authored, of which 270 were auto-generated by stay-context; 752 PRs reviewed

IGAWorks (Adpopcorn)

2021.07 - 2022.09

FE Engineer

New Adpopcorn SSP Console

Problem The existing Next.js SSR structure did not suit a console: it was built with SSR, but it carried a lot of legacy code and did not take advantage of SSR

Decision Judged that CSR offered more benefits; moved deployment from ECS to AWS Amplify

Action Built a new CRA console replacing the existing Next.js-based console (2022.04-2022.09), with React-Query + Recoil state management and Recharts dashboards

Result Every commit and push is built and deployed automatically by Amplify

Conpick

Problem The service renders HTML received from the server, so the project was closer to running a server than to building separate frontend code

Decision Because the project was closer to running a server, chose NestJS for its strong TypeScript support

Action Implemented news-content HTML conversion, a news template class and ad insertion with NestJS (2022.08-2022.09); set up ECS + CodePipeline CI/CD, writing the buildspec, appspec and task definition files

Result Every commit and push is built and deployed automatically through the dev and prod CodePipeline pipelines

Adpopcorn homepage

Problem The company's rebranding and business split called for new pages

Decision Chose Gatsby because the site is static

Action Built the homepage with Gatsby + Tailwind + i18n (2022.08-2022.09) — Monetize, Adserver and Offerwall pages, an inquiry form with an English page, and GA

Code-quality tooling

Problem There were no code rules before, so the code was disorderly

Decision To secure code stability and consistency before anything reached Git, enforced the checks at commit time with Husky + lint-staged

Action Applied ESLint + Prettier + Husky + lint-staged

Result Fewer CI/CD build issues and consistent code, which made collaboration easier

TypeScript

Problem There were problems with automatic type coercion

Decision Chose TypeScript for type safety

Action Repositories I initialized were written in TypeScript from the first commit

Result Adpopcorn SSP Console repository: 625 ts/tsx files and 0 js files

Zimssa (partner app)

2020.12 - 2021.07

FE Developer

Partner app

Problem	The app was built not natively but on Cordova, which was no longer being updated or developed
Decision	Decided to create a new TypeScript-based project and move to it, for type safety and easier collaboration
Action	Developed the Cordova hybrid app (Android); started the JS→TS transition by creating a CRA TypeScript-template project and updating dependencies; dark mode; applied ESLint + Prettier + Husky + lint-staged
Result	Every commit runs eslint --fix and prettier --write automatically through lint-staged; src holds 422 js/jsx and 31 ts/tsx files, so the TypeScript transition had only just begun

Partner web v2 component system

Problem	Each developer designed UI in their own way
Decision	Chose Atomic Design to bring consistency through a single structure and to reuse components, and Storybook for testing
Action	Built an Atomic Design component system (atoms, molecules, organisms) with Storybook
Result	123 Storybook story files

Design 2.0 renewal

Problem	Dark mode had to be supported
Decision	Lists had many items, which made loading and rendering slow and caused stutter, so decided to apply Lazy-Loading to improve both UI/UX and performance
Action	Owned the partner app's design 2.0 renewal and applied Lazy-Loading
Result	Applying Lazy-Loading improved initial loading

Personal projects



claude-administer

Claude Administer (2026.10) — A macOS menu-bar app that gathers skills, commands, agents, hooks, plugins, MCP servers, marketplaces and rules across global and project scope into a single list · Electron · React · TypeScript · Vite

https://github.com/maeng2418/claude-administer



bookmark-pro

BookmarkPro (2025.07 - 2025.12) — A modern bookmark manager for managing bookmarks efficiently on the web and in a browser extension, structured as a Turborepo monorepo · Next.js · Supabase · shadcn/ui · Manifest V3

https://github.com/maeng2418/bookmark-pro



Maeng-Design

Maeng Design (2021.08 - 2022.03) — React UI library — npm package maeng-design, with a documentation site · React · TypeScript · Emotion · Storybook

https://maeng-design-docs.netlify.app/



SEOUL13_edition2

SEOUL13 Edition2 (2020.09 - 2021.06) — SEOUL13 meeting-room scheduling web application, Edition2 · React · TypeScript · Next.js · Jest · Express · MySQL · Team project

Skills

Frontend	TypeScript · React · Next.js · Vue · Nuxt
State and forms	React Query · Recoil · React Hook Form · Zod
UI and architecture	MUI · Tailwind · Emotion · Storybook · FSD
Build and test	Playwright · MSW · Module Federation
Infrastructure and CI	GitHub Actions · Docker · Kubernetes · S3 · CloudFront · Sentry
AI developer tools	Claude Code · Claude Agent SDK · MCP · Cursor

maeng2418@gmail.com

Documents and standards

[RFC] React migration strategy for the partner onboarding admin · [TechSpec] Docker/Kubernetes and · path-based routing · [RFC] Form and data-transformation conventions for overseas accommodations (2026) · Claude Plugin structure and distribution strategy (2026.04)

Education and activities

Seoul National University of Science and Technology · B.S. in Computer Science and Engineering	2015.03 - 2021.02
Woowa Tech Camp, cohort 3	2020.07 - 2020.08
Researcher, Railway Management Policy Research Institute	2019.10 - 2020.06
DevelUp (goorm), cohort 1	2019.09 - 2019.11
Frontend intern, Dable (Blokee)	2019.08 - 2019.09

maeng2418@gmail.com